Generative data augmentation for a 4-class food classifier

Assignment A2 - extends the A1 baseline (bread / drinks / fruit / snacks)

Student: Ghani | Course project | November 2026

1. Summary

In A1 a small CNN was trained on 840 own photos of four food categories. In A2 the dataset is extended with extra real photos and with synthetic images produced by a recent text-to-image method (TA-TiTok + MaskGen), a stronger classifier (ConvNeXt-Tiny) is compared against the A1 baseline, an ablation over the training data is run, the models are explained (Grad-CAM, nearest neighbours), and everything is packaged in a Gradio web application. Main results:

- **Data.** +300 real photos from Food-101 (bread, snacks) and 400 synthetic images (100 per class); synthetic images make up 26 % of the training set (limit: 30 %) and are used for training only.
- **Accuracy.** The A1 baseline and both ConvNeXt-Tiny variants reach 100 % on the 180-image test set (macro-F1 = 1.0). The test set is saturated, so **this experiment cannot show whether synthetic data helps**; we do not claim that it does.
- **Training behaviour.** The pretrained ConvNeXt reaches ~100 % validation accuracy within three epochs and trains smoothly; the baseline needs about ten epochs and its validation curve is noisy.
- **Not done.** ArtAug (a LoRA for FLUX.1-dev) could not be run on the free Colab T4 GPU, so the ablation has two variants instead of three. A harder, independent test set is the main open item (Section 7).
- **App.** Upload/webcam input, class probabilities, Grad-CAM, 5 nearest real photos, latency and environment display, and a "Generate similar" button that shows pre-generated images with a visible SYNTHETIC banner.

2. Dataset update

The own dataset (A1) consists of phone photos, usually one item on a dark cloth or a white table (Fig. 1). The train/validation/test split of A1 was kept unchanged, so results remain comparable. The new data is used only in the training set; validation and test contain only own real photos.

Class	Train (own)	Validation	Test	Extra real	Synthetic (raw)	Synthetic (ArtAug)
bread	210	45	45	150	100	0
drinks	210	45	45	0	100	0
fruit	210	45	45	0	100	0
snacks	210	45	45	150	100	0
Total	**840**	**180**	**180**	**300**	**400**	**0**

Table 1. Dataset after the A2 update (counts of images).

Extra real photos. From Food-101 we took 150 random images per class (seed 42): bread from *garlic_bread* and *bruschetta*; snacks from *nachos, french_fries, onion_rings, churros*. Food-101 has no suitable classes for drinks and fruit, so these two classes received no extra real data. As a result the real training data is unbalanced (bread and snacks: 360 each, drinks and fruit: 210 each). The synthetic images are balanced (100 per class) and partly compensate this.

Mixing rule. Real training data = 840 + 300 = 1,140 images. Synthetic data is capped at 30 % of the final training set, which would allow up to 488 images; all 400 are used, i.e. 400 / 1,540 = 26 %.

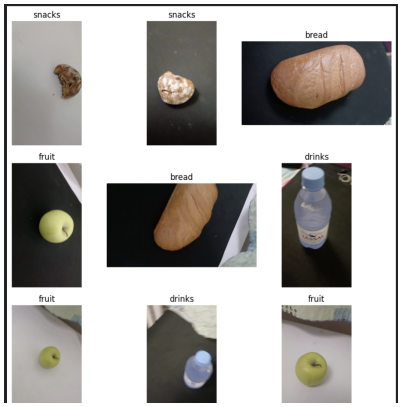


Figure 1. Examples from the own dataset (class shown above each photo).

3. Methods

3.1 Synthetic images: TA-TiTok + MaskGen

TA-TiTok is a text-aware one-dimensional image tokenizer: instead of a 2-D grid of latent patches it represents an image with a short 1-D sequence of tokens (the checkpoint name *bl32* indicates 32 tokens). MaskGen is a masked generative model that predicts these tokens iteratively, conditioned on CLIP text features; it is trained on open data. Both are released in the open-source ByteDance *1d-tokenizer* repository [1,2], which makes the pipeline cheap enough to run on a T4 GPU. We used the checkpoints `turkeyju/tokenizer_tatitok_bl32_vae` and `turkeyju/generator_maskgen_kl_xl` with 32 sampling iterations, classifier-free guidance 3.0 and aesthetic score 6.5.

For every class we wrote 10 prompts that vary the item, viewpoint, light and background (for example: "*a loaf of sourdough bread on a wooden board, side view*", "*an iced coffee in a plastic cup, daylight*", "*a bowl of mixed fresh fruit on a kitchen counter*", "*a pack of crackers next to a drink, low light*") and sampled 10 seeds per prompt, giving 100 images per class. The prompts deliberately describe scenes that differ from our own photos (wooden boards, kitchens, outdoor tables, low light) to add variety that the real data lacks. The class label of a synthetic image is the class of its prompt. We inspected 12 random images per class visually; the review showed some off-class or unrealistic samples (for example, a sofa scene generated for the "snacks" prompt about cookies on a sofa), which is why such images must be removed and why labels of synthetic data should not be trusted blindly.

3.2 ArtAug (not run)

ArtAug [3] improves the aesthetic quality of a text-to-image model through interaction between the generator and an image-understanding model: the aesthetic judgement of a large vision-language model (Qwen2-VL-72B) is distilled into a LoRA for FLUX.1-dev, which then produces more attractive images at generation time. It is therefore a modification of the generator, not a post-processing filter. FLUX.1-dev has about 12 billion parameters; on the free Colab T4 (16 GB GPU memory, ~12 GB RAM) we could not complete the setup within the time available. The ablation variant "real + synthetic + ArtAug" is consequently missing, and the effect of ArtAug is **not measured** in this report. The notebook handles this case automatically (the third variant is trained only if `synthetic/artaug` exists).

3.3 Classifiers and training protocol

- **A1 baseline CNN:** three Conv-ReLU-MaxPool blocks (32/64/128 filters), global average pooling, dropout 0.3, dense 64, softmax; trained on the 840 own photos as in A1 (Adam, up to 15 epochs, early stopping on validation loss with patience 3).
- **ConvNeXt-Tiny** (ImageNet weights) [4] with global average pooling, dropout 0.3 and a softmax head. Stage 1: backbone frozen, 10 epochs, learning rate 1e-3; stage 2: last 40 layers unfrozen, 10 epochs, learning rate 1e-5. Early stopping as above.
- Common settings: 224x224 input, batch 32, seed 42, augmentation (horizontal flip, rotation 0.1, zoom 0.1, contrast 0.1). The best model for the demo is chosen on the **validation** set only; the test set is used once for reporting.
- **Ablation variants of ConvNeXt:** (a) real = 840 own + 300 extra real; (b) real + synthetic (400 raw images); (c) real + synthetic + ArtAug (not run).

4. Experiments

Model	Train real	Train synth.	Synth. share	Acc. (top-1)	Top-2	Macro P	Macro R	Macro F1
A1 baseline CNN	840	0	0 %	1.00	1.00	1.00	1.00	1.00
ConvNeXt-Tiny [real]	1,140	0	0 %	1.00	1.00	1.00	1.00	1.00
ConvNeXt-Tiny [real + synth]	1,140	400	26 %	1.00	1.00	1.00	1.00	1.00
ConvNeXt-Tiny [real + synth + ArtAug]	-	-	-	not run	-	-	-	-

Table 2. Results on the real test set (180 images, 45 per class). All three trained models classify every test image correctly.

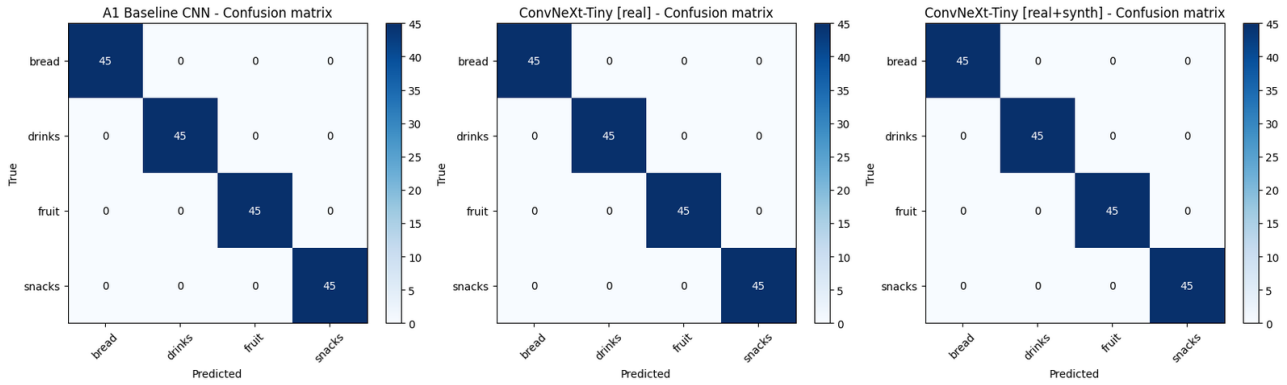


Figure 2. Confusion matrices on the test set: A1 baseline, ConvNeXt-Tiny [real], ConvNeXt-Tiny [real + synth].

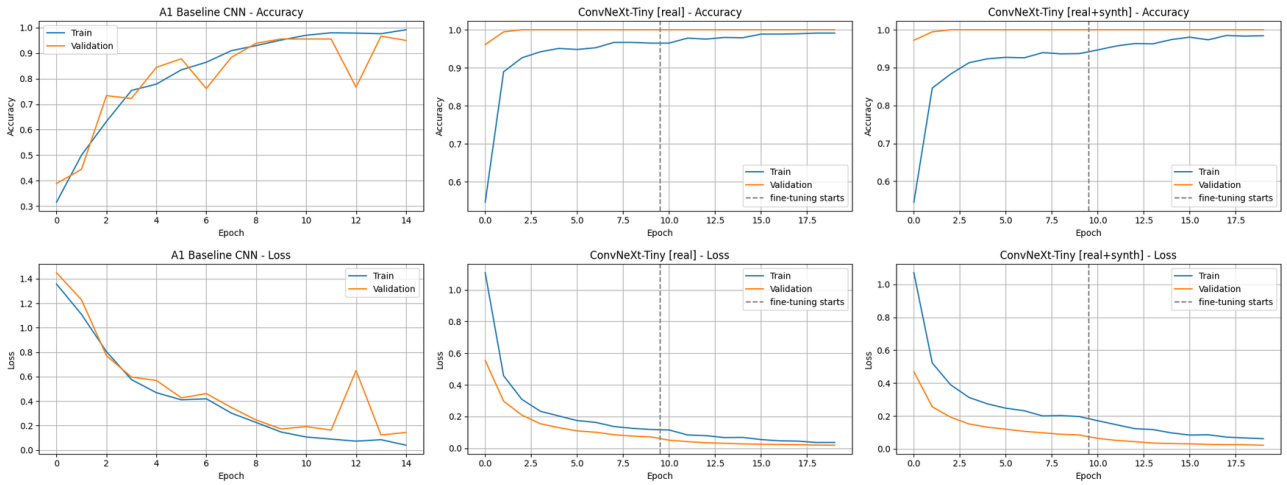


Figure 3. Training curves (top: accuracy, bottom: loss) for the baseline (left), ConvNeXt [real] (middle) and ConvNeXt [real + synth] (right). The dashed line marks the start of fine-tuning.

What the results show. On test accuracy and F1 the three models are indistinguishable: 180 of 180 correct each. With 180 test images a perfect score only supports the statement that the true error rate is below roughly 1.7 % ("rule of three", 3/n), so the test set cannot rank the models. The curves do show differences in training behaviour. The pretrained ConvNeXt reaches ~100 % validation accuracy by the third epoch and its validation loss falls smoothly to about 0.02. The baseline climbs slowly (about 0.95 validation accuracy after ten epochs) and is unstable: validation accuracy drops to about 0.77 at epoch 12 before recovering. Early stopping restores the best-validation weights, which is why its test result is still perfect. Adding synthetic data did not change validation or test results; it only made the training set harder to fit (final training loss about 0.06 against 0.035 without synthetic data, accuracy 0.98 against 0.99), which is expected for a more diverse, partly imperfect data source. On validation the two ConvNeXt variants tie on macro-F1, so the tie is broken by validation loss, and the [real] variant was selected for the demo.

Why we cannot conclude more. (i) The test set is saturated. (ii) Our nearest-neighbour search (Section 5) shows that most test photos have a near-duplicate in the training set (same object, same background, cosine similarity 0.92-0.99), so the benchmark measures memorisation of the scene at least as much as generalisation. (iii) Each variant was trained once (seed 42), so there is no estimate of run-to-run variance.

Under these conditions the honest conclusion is: *synthetic data from TA-TiTok + MaskGen did no harm, but our test set is too easy to show a benefit.*

5. Explainability

Grad-CAM. For the predicted class we compute the gradient of the class score with respect to the last ConvNeXt feature map, average it over space to obtain channel weights, and form a ReLU-ed weighted sum of the maps [5]. The heat map is up-sampled and overlaid on the photo (Fig. 4). On the eight examples shown (random correctly classified test photos) the high-activation region lies on the food item itself - the loaf, the cookie, the bottle - and not on the cloth or table, with confidences between 0.96 and 1.00. This is reassuring, but since no test image is misclassified we could not use Grad-CAM to study failure cases.

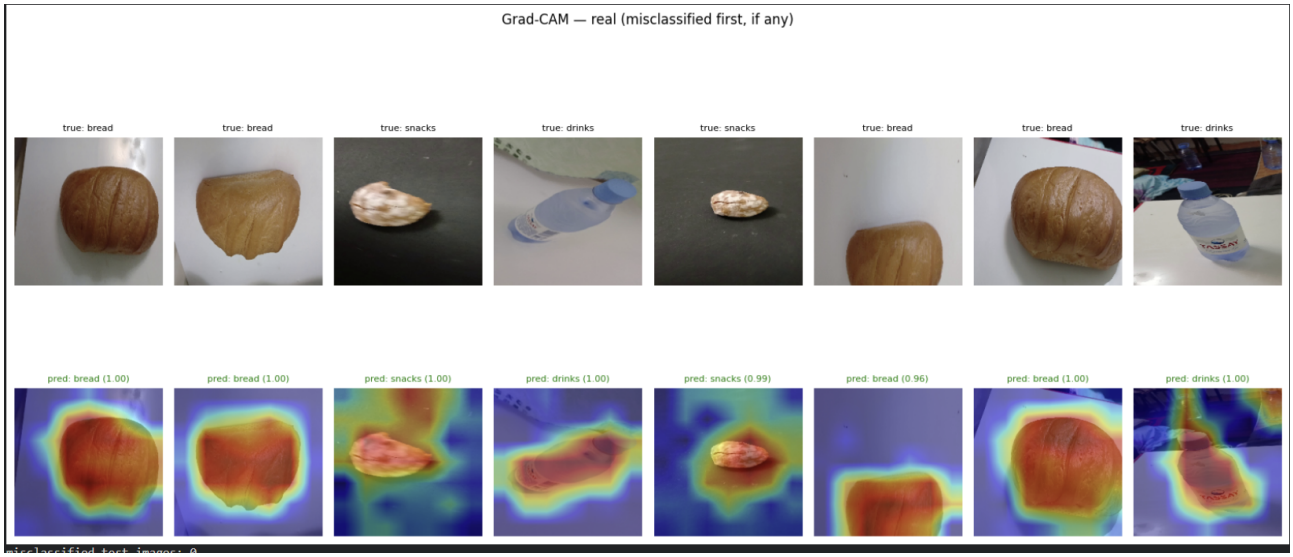


Figure 4. Grad-CAM for ConvNeXt-Tiny [real]: test photos with true labels (top) and predictions with heat maps (bottom).

Nearest neighbours. The backbone's pooled feature vector is used as an embedding; for a query we retrieve the most similar real training photos by cosine similarity (Fig. 5). Retrieval precision on the test set (query = test photo, gallery = 840 real training photos):

Embedding	P@1	P@5	P@10
ImageNet backbone (no fine-tuning)	0.9944	0.9944	0.9944
Fine-tuned ConvNeXt-Tiny [real]	1.0000	1.0000	1.0000

Table 3. Precision@k of class retrieval.

Even the untouched ImageNet backbone retrieves the correct class for 99.4 % of the queries, and the retrieved photos look like further frames of the same object (Fig. 5). This is the evidence behind the near-duplicate remark in Section 4 and shows why the task is easy for any reasonable feature extractor.

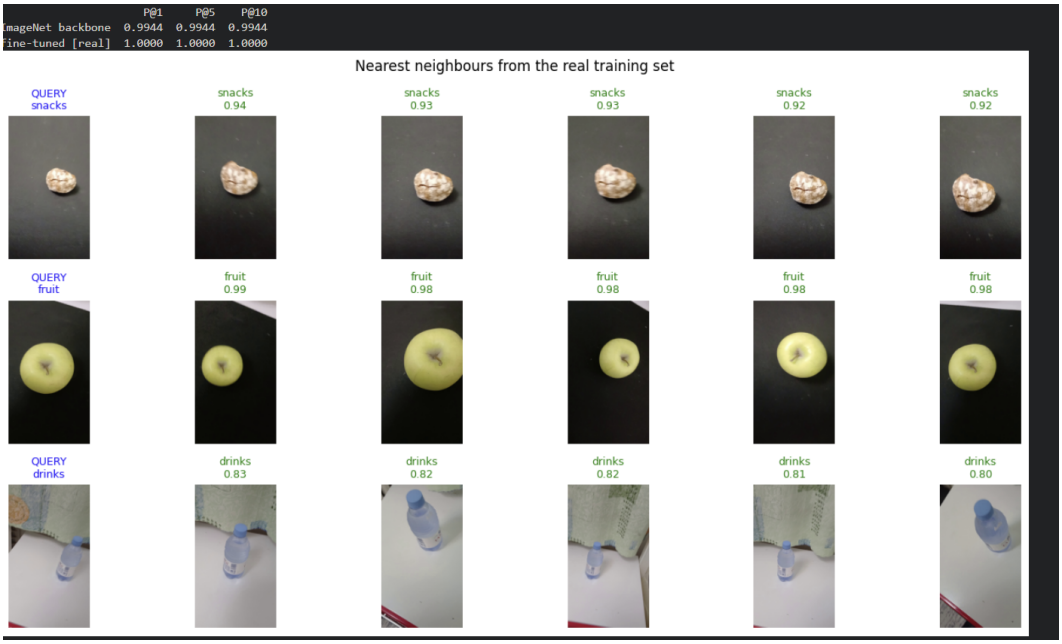


Figure 5. Five nearest real training photos for three test queries (green = same class, similarity shown under each image).

6. Web application

The Gradio application (notebook cell "app") loads the selected model and offers: image upload or webcam capture; class probabilities; Grad-CAM overlay; the five nearest real training photos with similarity scores; per-request latency, running average and environment information; and a **Generate similar** button that displays four pre-generated synthetic images of the predicted class (from the ArtAug folder if available, otherwise the raw TA-TiTok + MaskGen set). Every such image is stamped with a red *SYNTHETIC / AI-GENERATED* banner and a text note. The generation itself is done offline, not on demand, because the generator does not fit comfortably in the app's runtime.

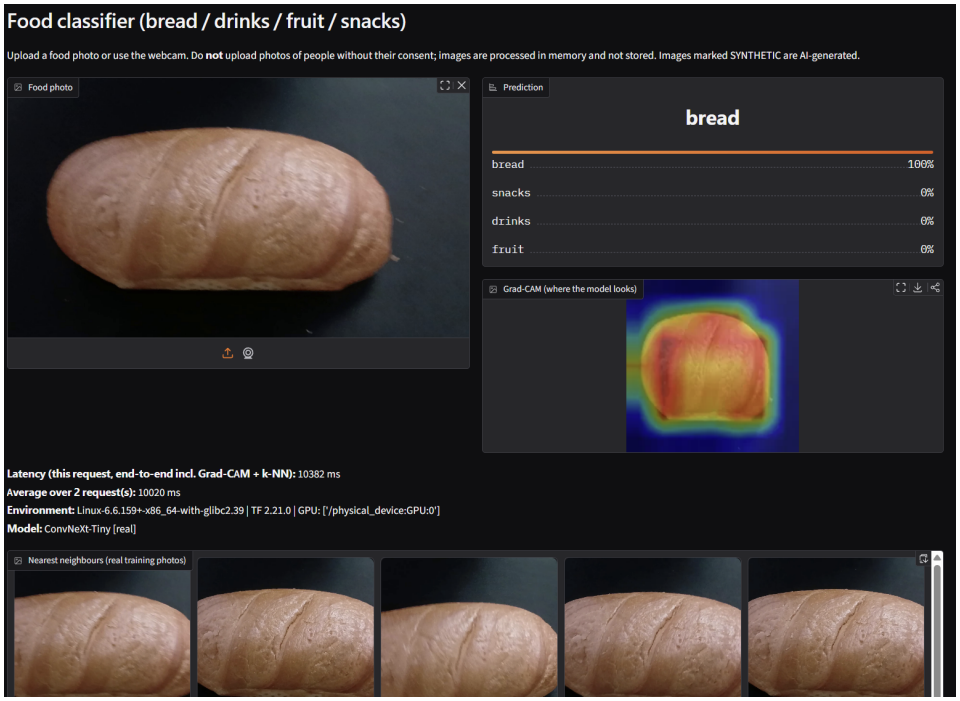


Figure 6. The application classifying a bread photo: probabilities, Grad-CAM, latency/environment and nearest neighbours.

Latency and environment. Environment: Linux 6.6 (x86_64), Python 3.13, TensorFlow 2.21.0, NVIDIA T4 GPU (Google Colab). In the app a complete request - ConvNeXt forward pass, Grad-CAM, embedding and nearest-neighbour search - took 10.4 s for the screenshot request and 10.0 s on average over two requests. This is too slow for interactive use. The notebook log shows repeated *tf.function retracing* warnings, which suggests that the eager execution path is recompiled on many calls; we have not verified this nor optimised it. Likely fixes are wrapping the model and Grad-CAM in `tf.function` with a fixed input

signature and computing the embedding in the same pass. A model-only forward-pass benchmark per model and device is written by the notebook to `results/latency.csv`.

7. Ethics, limitations and future work

Marking synthetic content. Synthetic images are kept in a separate folder from real ones, never enter the validation or test sets (otherwise metrics would be inflated), and are always labelled as AI-generated when shown in the app. **Privacy.** The data contain food items only; the app warns users not to upload photos of people without consent, and uploaded images are processed in memory and not stored. **Provenance and licences.** The own photos were taken by the author. Food-101 is a public research dataset made of user-contributed images and was used here for non-commercial coursework; the generators are open-source models whose terms apply to the produced images. **Bias.** Food-101 is dominated by Western dishes and covers only two of our four classes. Our prompts encode our own idea of what "bread" or "snacks" look like (loaves, baguettes, chips, cookies), so the synthetic data inherits both our assumptions and the generator's biases; the classifier may fail on other cuisines. Synthetic labels come from prompts and were checked only by visual sampling.

Limitations.

- The test set is saturated (100 %) and likely contains near-duplicates of training scenes, so the benefit of synthetic data cannot be assessed.
- ArtAug was not run; the ablation has two instead of three variants.
- One training run per variant (seed 42), no confidence intervals; four classes, mostly single items on simple backgrounds.
- App latency (~10 s per request) is not suitable for real-time use.

Future work. (1) Build an independent hard test set - new photos with different light, backgrounds and several items per image - and report metrics on both test sets; a version of the test set perturbed with darkening, blur, noise and random crops is a quick substitute, but only new photos are a true generalisation test. (2) Train with several seeds and report mean and spread. (3) Run ArtAug on a larger GPU and complete the three-way ablation. (4) Reduce app latency with compiled inference. (5) Collect extra real data for drinks and fruit.

8. How to reproduce

The repository contains the notebook `Ghani_FoodClassification_A2.ipynb`, the generation notebook for the synthetic images, a README and the `results/` folder (tables, figures, `metrics.json`, saved models). Steps (Google Colab, GPU = T4):

- Generate the synthetic images with the generation notebook (clones *bytedance/1d-tokenizer*, writes `synthetic/raw/<class>/`) and zip them as `synthetic.zip`.
- Put `dataset.zip`, `extra_real.zip` (folders bread/drinks/fruit/snacks, the last two may be empty) and `synthetic.zip` on Google Drive.
- In the main notebook: mount Drive, copy the three archives to `/content`, then run the cells in order: data loading, dataset table, baseline, ConvNeXt experiments, comparison, Grad-CAM, nearest neighbours, latency, saving, application.

Local run instructions for the application are given in the README. Random seed 42 is fixed for splits, shuffling and initialisation of the new layers; GPU kernels may still cause small numerical differences.

References

[1] ByteDance, *1d-tokenizer* (TA-TiTok, MaskGen), https://github.com/bytedance/1d-tokenizer.
[2] D. Kim et al., Democratizing Text-to-Image Masked Generative Models with Compact Text-Aware One-Dimensional Tokens, arXiv:2501.07730, 2025 (please verify the reference against the course material).
[3] ArtAug: Enhancing Text-to-Image Generation through Synthesis-Understanding Interaction, arXiv:2412.12888; code: ModelScope DiffSynth-Studio; LoRA: ECNU-CILab/ArtAug-lora-FLUX.1dev-v1 (Hugging Face).
[4] Z. Liu et al., A ConvNet for the 2020s (ConvNeXt), CVPR 2022.
[5] R. Selvaraju et al., Grad-CAM: Visual explanations from deep networks via gradient-based localization, ICCV 2017.
[6] L. Bossard et al., Food-101 - Mining discriminative components with random forests, ECCV 2014.